

Programmation fonctionnelle 2
Preuves et programmes
CM6 : Réécriture, égalité (1ère partie)

Pierre Rousselin

Université Sorbonne Paris Nord

février 2026

Réécritures et preuve papier

L'égalité (première partie)

rewrite, continued

Égalité entre constructeurs

Où en sommes-nous ?

```
Print list. (*
Inductive list (A : Type) : Type :=
  nil : list A / cons : A -> list A -> list A. *)
From Coq Require Import List.
Import ListNotations.
Locate "[ ]". (* nil *)
Locate "_ :: _". (* Notation "x :: y" := (cons x y) *)
Fixpoint app [A : Type] (l1 l2 : list A) :=
  match l1 with
  | [] => l2
  | h :: t => h :: (app t l2)
  end.
Notation "l1 ++ l2" := (app l1 l2).
Lemma app_assoc [A : Type] (l1 l2 l3 : list A) :
  l1 ++ (l2 ++ l3) = (l1 ++ l2) ++ l3.
Proof.
  induction l1 as [| h t IHt].
  - simpl. reflexivity.
  - simpl. rewrite IHt. reflexivity.
Qed.
```

Règles de réécritures de base

- Problème pour un humain : pas évident de savoir ce que va faire `simpl` dans une preuve lue sans avoir Rocq sous la main.
- De plus, dans une preuve sur papier, on pourrait écrire « par calcul », mais c'est mieux d'explicitier un peu plus pour le lecteur.
- Pour relier les deux mondes, on va utiliser des réécritures explicites avec `rewrite`,
- en se donnant comme règles de réécritures initiales les différents cas dans le `match` de la définition.
- Ici,

```
Lemma app_nil_1 {A : Type} (l : list A) : [] ++ l = l.
```

```
Proof. simpl. reflexivity. Qed.
```

```
Lemma app_cons_1 {A : Type} (h : A) (q l : list A) :  
  (h :: q) ++ l = h :: (q ++ l).
```

```
Proof. simpl. reflexivity. Qed.
```

Règles de réécriture de base (2)

- Pour la longueur :

```
Lemma length_nil {A : Type} : length (@nil A) = 0.
```

```
Proof. simpl. reflexivity. Qed.
```

```
Lemma length_cons {A : Type} (h : A) (q : list A) :  
  length (h :: q) = S (length q).
```

```
Proof. simpl. reflexivity. Qed.
```

- Pour l'addition des entiers :

```
Lemma add_0_l (n : nat) : 0 + n = n.
```

```
Proof. simpl. reflexivity. Qed.
```

```
Lemma add_succ_l (n m : nat) : (S n) + m = S (n + m).
```

```
Proof. simpl. reflexivity. Qed.
```

Nouvelle preuve avec réécritures explicites

```
Lemma app_assoc' {A : Type} (l1 l2 l3 : list A) :  
  (l1 ++ l2) ++ l3 = l1 ++ (l2 ++ l3).
```

Proof.

```
induction l1 as [| h q IH].
```

```
- rewrite app_nil_l. (* but : l2 ++ l3 = [] ++ (l2 ++ l3) *)  
  rewrite app_nil_l. (* but : l2 ++ l3 = l2 ++ l3 *)  
  reflexivity.  
- rewrite app_cons_l.  
  (* but : (h :: (q ++ l2)) ++ l3 = (h :: q) ++ (l2 ++ l3) *)  
  rewrite app_cons_l.  
  (* but : h :: ((q ++ l2) ++ l3) = (h :: q) ++ (l2 ++ l3) *)  
  rewrite app_cons_l.  
  (* but : h :: ((q ++ l2) ++ l3) = h :: (q ++ (l2 ++ l3)) *)  
  rewrite IH.  
  (* but : h :: (q ++ (l2 ++ l3)) = h :: (q ++ (l2 ++ l3)) *)  
  reflexivity.
```

Qed.

Petite remarque sur les `rewrite`

- ▶ Il y a, pour des raisons historiques, plusieurs versions différentes de la tactique `rewrite` en Rocq.
- ▶ On essaie ici la version dite `ssreflect`, jugée plus robuste (et qui devrait devenir la version par défaut prochainement).
 - ▶ Pour Coq (versions 8.20, 8.19, 8.18)
`From Coq Require Import ssreflect.`
 - ▶ Pour Rocq
`From Corelib Require Import ssreflect.`
- ▶ `ssreflect` (pour *small scale reflection*, réflexion de petite échelle) est un langage de tactiques développé par George Gonthier pour la preuve du théorème des 4 couleurs la preuve du théorème de Feit-Thompson. Pour l'instant on garde surtout `rewrite`.

Enchaîner les rewrite

- Syntaxe :

```
rewrite règle1 règle2 ...
```

- Lemma app_assoc' {A : Type} (l1 l2 l3 : list A) :
 (l1 ++ l2) ++ l3 = l1 ++ (l2 ++ l3).

Proof.

```
induction l1 as [| h q IH].
```

```
- rewrite app_nil_l app_nil_l. reflexivity.
```

```
- rewrite app_cons_l app_cons_l app_cons_l IH. reflexivity.
```

Qed.

Enchaîner les rewrite

- Syntaxe :

```
rewrite règle1 règle2 ...
```

- **Lemma** app_assoc' {A : Type} (l1 l2 l3 : list A) :
 (l1 ++ l2) ++ l3 = l1 ++ (l2 ++ l3).

Proof.

```
induction l1 as [| h q IH].
```

```
- rewrite app_nil_l app_nil_l. reflexivity.
```

```
- rewrite app_cons_l app_cons_l app_cons_l IH. reflexivity.
```

Qed.

- rewrite au maximum n fois avec la même règle :

```
rewrite n!règle1 ...
```

- **Lemma** app_assoc' {A : Type} (l1 l2 l3 : list A) :
 (l1 ++ l2) ++ l3 = l1 ++ (l2 ++ l3).

Proof.

```
induction l1 as [| h q IH].
```

```
- rewrite 2!app_nil_l. reflexivity.
```

```
- rewrite 3!app_cons_l IH. reflexivity.
```

Qed.

Sur papier

Démonstration.

On raisonne par induction sur l_1 .

- Dans le cas où $l_1 = []$, $([] ++ l_2) ++ l_3 \stackrel{\text{app_nil-1}}{=} l_2 ++ l_3$,

tandis que, $[] ++ (l_2 ++ l_3) \stackrel{\text{app_nil-1}}{=} l_2 ++ l_3$.

- Dans le cas où $l_1 = h :: t$, où t satisfait

$$(t ++ l_2) ++ l_3 = t ++ (l_2 ++ l_3), \quad (\text{IH})$$

on a :

$$\begin{aligned} ((h :: t) ++ l_2) ++ l_3 &\stackrel{\text{app_cons-1}}{=} (h :: (t ++ l_2)) ++ l_3 \\ &\stackrel{\text{app_cons-1}}{=} h :: (t ++ (l_2 ++ l_3)) \\ &\stackrel{\text{IH}}{=} h :: ((t ++ l_2) ++ l_3), \end{aligned}$$

tandis que :

$$(h :: t) ++ (l_2 ++ l_3) \stackrel{\text{app_cons-1}}{=} h :: (t ++ (l_2 ++ l_3)).$$



Prouver une égalité par les deux bouts

- ▶ Ce qui n'était pas visible dans la preuve en Rocq
- ▶ est qu'on a travaillé successivement sur chaque côté de l'égalité à prouver
- ▶ qu'on a réécrits en des termes identiques,
- ▶ en utilisant les règles de réécritures de gauche vers la droite.
- ▶ En général, c'est le plus simple.

Réécritures et preuve papier

L'égalité (première partie)

rewrite, continued

Égalité entre constructeurs

Égalité en OCaml

- ▶ Égalité dite « structurelle » avec `=`¹

```
let b1 = (3 :: 4 :: [] = (2 + 1) :: (2 + 2) :: [])  
(* val b1 : bool = true *)  
let b2 = (3 :: [] = 1 :: 2 :: [])  
(* val b2 : bool = false *)
```

- ▶ Deux valeurs (d'un même type, sinon erreur à la compilation) sont égales si elles sont construites avec le même constructeur, et que les arguments des constructeurs sont égaux
- ▶ etc, si besoin jusqu'aux types primitifs où l'égalité est l'égalité usuelle.
- ▶ Diségalité avec `<>`

```
let b3 = (3 :: [] <> 1 :: 2 :: [])  
(* val b3 : bool = true *)
```
- ▶ Égalité (resp. diségalité) « physique » `==` (resp. `!=`) à ne pas utiliser dans ce cours (utile juste en impératif).

1. Ici les parenthèses extérieures sont juste là pour faire la différence entre la première égalité qui attache un nom à une valeur et la seconde qui compare deux listes.

Égalité en OCaml (2)

- ▶ Remarquer que l'égalité d'OCaml est à valeurs dans `bool`
- ▶ et est évaluée comme n'importe quel autre opérateur.
- ▶ Mais il n'y a pas d'égalité entre
- ▶ fonctions,
- ▶ variants dont un constructeur a un champ fonction, etc.
- ▶ `let b = ((fun x -> x) = (fun y -> y))`
(Exception: Invalid_argument "compare: functional value". *)*
- ▶ On est dans un cas assez « simple » d'égalité décidable entre valeurs.

En Rocq

- ▶ L'égalité usuelle, notée $=$ n'est pas primitive et vit dans `Prop`.
- ▶ Dans le prélude :

```
Inductive eq {A : Type} (x : A) : A -> Prop :=  
  eq_refl : x = x.
```

- ▶ Dur à avaler, pour l'instant, on retient que pour chaque type `A`, `@eq A` est une relation binaire sur les éléments de `A`, c'est-à-dire une fonction de type `A -> A -> Prop`
- ▶ avec un seul constructeur `eq_refl` qui, pour tout `x` de type `A` prouve que `x = x`.
- ▶ C'est donc la plus petite relation réflexive sur chaque type.

eq_refl pour prouver une égalité

- ▶ Notre reflexivity est en fait (dans ce cas) un `exact eq_refl`.
`Lemma bar1 : 2 = 2. Proof. exact eq_refl. Qed.`
- ▶ Les arguments d'`eq_refl` sont en fait implicites :
`Lemma bar2 : 2 = 2. Proof. exact (@eq_refl nat 2). Qed.`
- ▶ Les termes sont toujours considérés à réductions (et conversions) près
`Lemma bar3 : 1 + 1 = 2. Proof. exact eq_refl. Qed.`
`Lemma bar4 : 1 + 1 = 2.`
`Proof. exact (@eq_refl nat (1 + 1)). Qed.`
- ▶ `Lemma bar5 : 1 + 1 = 2.`
`Proof. exact (@eq_refl nat (0 + 1 + 0 + 1)). Qed.`

`simpl. reflexivity.`

- ▶ Vu que les termes sont toujours considérés à réductions et conversions près,
- ▶ il n'est en fait pas utile d'utiliser `simpl` avant `reflexivity`
- ▶ si ce n'est bien sûr pour voir Rocq réduire les termes, ce qui est certainement utile du point de vue de l'utilisateur.

Utiliser une égalité

- ▶ On rentrera dans les détails lors d'un autre cours
- ▶ mais pour l'instant, il suffit de voir que Rocq génère pour nous un éliminateur pour l'égalité² :
- ▶ **Check** `eq_ind`. (*
 `eq_ind : forall (A : Type) (x : A) (P : A -> Prop),`
 `P x -> (forall y : A, x = y -> P y) *`*)
- ▶ Si $x = y$, alors toute propriété qui est satisfaite par x , l'est aussi par y .
- ▶ Donc si j'ai une preuve H que $x = y$, et que je dois prouver $P y$, alors, en appliquant `eq_ind` il suffit de le prouver pour $P x$.
- ▶ C'est à peu de chose près ce que fait pour nous la tactique `rewrite H`.
- ▶ On continuera sur l'égalité (règles de déduction naturelle, symétrie, lien avec l'égalité de Leibniz, etc) dans une autre cours.

2. avec un `match` dépendant un peu dur à comprendre

Réécritures et preuve papier

L'égalité (première partie)

rewrite, continued

Égalité entre constructeurs

De droite à gauche

- ▶ On a vu les égalités comme des règles de réécriture de gauche à droite.
- ▶ On peut aussi les considérer de droite à gauche.
- ▶ Syntaxe :

```
rewrite -règle ...
```

- ▶ Exemple :

```
Lemma rewrite_rl (n : nat) : 2 = n -> n + 4 = 6.
```

```
Proof.
```

```
  intros H. (* H : 2 = n *)
```

```
  rewrite -H. (* changer n en 2 *)
```

```
  reflexivity.
```

```
Qed.
```

- ▶ Si on veut combiner « de droite à gauche » avec « plusieurs fois », on fait comme ça :

```
Lemma rewrite_rl2 (n m : nat) (f : nat -> nat) :  
  (forall x, x + 3 = f x) -> (f 0) + 1 = (f 1).
```

```
Proof.
```

```
  intros H. (* H : forall x, x + 3 = f x *)
```

```
  rewrite -2!H.
```

```
  reflexivity.
```

```
Qed.
```

Un terme bien précis

- ▶ La plupart des lemmes qu'on utilise avec `rewrite` sont quantifiés universellement.
- ▶ Si on ne précise rien, la réécriture n'aura pas forcément lieu là où on le veut.
- ▶ Plutôt que, par exemple, `rewrite add_comm`, on utilisera `rewrite (add_comm 2 n)` pour changer $2 + n$ en $n + 2$.

Exemple (purement pédagogique)

```
Check add_assoc. (*
add_assoc : forall n m p : nat, n + (m + p) = n + m + p *)
Check add_comm. (*
add_comm : forall n m : nat, n + m = m + n *)
Lemma rewrite_add_assoc (n m : nat) : (n + m) + 12 = (n + 3) + (m + 9).
Proof.
  rewrite -add_assoc. (* changer (n + m) + 12 en n + (m + 12) *)
  rewrite (add_assoc (n + 3) m 9). (* on précise les arguments *)
  rewrite -(add_assoc (n + 3) m 9). (* changé d'avis *)
  rewrite (add_comm m 9). (* changer m + 9 en 9 + m *)
  rewrite -add_assoc.
  rewrite (add_assoc 3 9 m).
  rewrite (add_comm (3 + 9) m).
  reflexivity.
Qed.
```

Réécritures et preuve papier

L'égalité (première partie)

rewrite, continued

Égalité entre constructeurs

Le prédécesseur d'un entier naturel

- ▶ Le prédécesseur d'un entier naturel est celui qui vient juste avant.
- ▶ On a un problème de définition en 0.
- ▶ En maths on dirait « c'est pas défini. »
- ▶ En Rocq (ou OCaml) on pourrait dire que ça donne une option.
- ▶ Mais pour pas s'embêter, on va plutôt dire que le prédécesseur de 0 est 0.
- ▶ **Definition** `pred (n : nat) :=`
 `match n with`
 `| 0 => 0 (* un peu arbitraire *)`
 `| S p => p`
 `end.`

Prédécesseur et successeur

- L'existence du prédécesseur entraîne l'injectivité du constructeur S de nat :

- **Lemma** `pred_succ (n : nat) : pred (S n) = n.`

Proof. `reflexivity. Qed.`

Lemma `succ_inj (n m : nat) : S n = S m -> n = m.`

Proof.

`intros H.`

`rewrite -(pred_succ n). (* remplacer n par (pred (S n)) *)`

`rewrite -(pred_succ m). (* remplacer m par (pred (S m)) *)`

`rewrite H.`

`reflexivity.`

`Qed.`

Même chose pour cons

- Pour la fonction `tl` (*tail*, queue de la liste) on fait comme pour `pred` :

```
Definition tl {A : Type} (l : list A) :=  
  match l with  
  | [] => [] (* un peu arbitraire *)  
  | _ :: t => t  
end.
```

- Pour la fonction `hd` (*head*, tête de la liste) on a un problème car dans le cas de la liste vide on ne voit pas trop ce qu'on pourrait retourner (le type `A` étant général).
- Astuce : ajouter à `hd` un paramètre pour la valeur « par défaut » dans ce cas.

```
Definition hd {A : Type} (default : A) (l : list A) :=  
  match l with  
  | [] => default  
  | h :: t => h  
end.
```

Même chose pour cons (2)

- On a les règles suivantes :

```
Lemma hd_cons {A : Type} (h : A) (t : list A) :  
  forall default, hd default (h :: t) = h.
```

```
Proof. reflexivity. Qed.
```

```
Lemma tl_cons {A : Type} (h : A) (t : list A) :  
  tl (h :: t) = t.
```

```
Proof. reflexivity. Qed.
```

- On en déduit :

```
Lemma cons_inj {A : Type} (h1 h2 : A) (t1 t2 : list A) :  
  h1 :: t1 = h2 :: t2 -> h1 = h2 /\ t1 = t2.
```

```
Proof.
```

```
  intros H.
```

```
  rewrite -(hd_cons h1 t1 h1) -(hd_cons h1 t2 h2).
```

```
  rewrite -(tl_cons h1 t1) -(tl_cons h2 t2).
```

```
  rewrite H.
```

```
  split; reflexivity.
```

```
Qed.
```

Injectivité des constructeurs

- ▶ En fait, on pourra toujours faire exactement ça, c'est-à-dire
- ▶ définir des fonctions réciproques (partielles) aux constructeurs
- ▶ pour prouver l'injectivité des constructeurs.
- ▶ Pour ne pas avoir à faire ce travail à chaque fois, il y a une tactique qui automatise ça : **injection**.

Tactique injection

(Ne pas avoir à renommer H avec injection
si le constructeur a un seul argument. *)*

Set Structural Injection.

Lemma succ_inj' (n m : nat) : S (S n) = S (S m)
-> n = m.

Proof.

intros H.
injection H. *(* H devient n = m *)*
exact H.

Qed.

Lemma cons_inj' {A : **Type**} (h1 h2 : A) (t1 t2 : list A):
h1 :: t1 = h2 :: t2 -> h1 = h2 /\ t1 = t2.

Proof.

intros H.
injection H as Hh Ht. *(* H est remplacé par Hh et Ht *)*
(Hh : h1 = h2 *)*
(Ht : t1 = t2 *)*
split; assumption.

Qed.

Égalité entre constructeurs différents

- ▶ Et entre constructeurs différents...
- ▶ Que dire, par exemple de $1 = 0$
- ▶ ou de $[] = [1; 4]$?
- ▶ On aimerait pouvoir prouver que c'est impossible.
- ▶ Encore une fois, c'est une conséquence de la possibilité de `match` sur les constructeurs.

Exemple : 1 <> 0

- ▶ On commence par définir une fonction qui vaut `True` pour un des constructeurs, `False` pour les autres. Ici, par exemple :

```
Definition is_succ (n : nat) :=  
  match n with S _ => True | 0 => False end.
```

- ▶ Ensuite, comme souvent, on peut vouloir des règles de réécriture pratiques :

```
Lemma is_succ_succ : forall n, is_succ (S n) = True.
```

```
Proof. intros n. reflexivity. Qed.
```

```
Lemma is_succ_zero : is_succ 0 = False.
```

```
Proof. reflexivity. Qed.
```

- ▶ Pour finir, on prouve, par exemple, que `1 <> 0` en remplaçant `False` par `is_succ 0`, puis `0` par `1` :

```
Lemma ne_1_0 : 1 <> 0.
```

```
Proof.
```

```
  intros H. (* H : 1 = 0, le but est False *)
```

```
  rewrite -is_succ_zero. (* le but est is_succ 0 *)
```

```
  rewrite -H. (* le but devient is_succ 1 *)
```

```
  simpl. (* qui vaut True *)
```

```
  exact I. (* qu'on sait prouver avec I *)
```

```
Qed.
```

discriminate

- ▶ Comme précédemment avec **injection**, ceci est automatisable.
- ▶ Et c'est ce que fait la tactique **discriminate**.
- ▶ Si H est une preuve d'une égalité, on utilise **discriminate** H pour prouver **False** et terminer la preuve
 - ▶ lorsque l'égalité est entre des constructeurs différents
 - ▶ après avoir éventuellement réduit les termes
 - ▶ et simplifié les constructeurs identiques.

- ▶ Exemple :

```
Lemma ne_1_0' : 1 <> 0.
```

```
Proof.
```

```
  intros H.
```

```
  discriminate H.
```

```
Qed.
```


discriminate, autres exemples

- Ici, on simplifie d'abord un peu et on aboutit à $0 = S\ n$.

```
Lemma exemple_disc1 (n : nat) : 2 <> S (S (S n)).
```

```
Proof.
```

```
  intros H.
```

```
  discriminate H.
```

```
Qed.
```

- Un autre exemple avec des listes (parce que pourquoi pas) :

```
Lemma exemple_disc2 (l1 : list nat) : 42 :: l1 <> [].
```

```
Proof. intros H. discriminate H. Qed.
```

- Ici, on a d'abord une réduction qui mène à `true = false`

```
Lemma exemple_disc_bool : (true && false) || true <> false.
```

```
Proof.
```

```
  intros H.
```

```
  discriminate H.
```

```
Qed.
```

- Et là, une réduction qui mène à $S\ (S\ n) = 0$:

```
Lemma exemple_disc3 (n : nat) : 2 + n <> 0.
```

```
Proof. intros H. discriminate H. Qed.
```

discriminate, contre-exemple

Dans l'autre sens, ça ne marche pas, Rocq ne sait pas quoi faire avec $n + 2$:

```
Lemma exemple_disc4 (n : nat) : n + 2 <> 0.
```

```
Proof.
```

```
  intros H.
```

```
  Fail discriminate H.
```

```
  (* has indeed failed with message :
```

```
    "Not a discriminable equality" *)
```

```
  rewrite add_comm in H. (* H : 2 + n = 0 *)
```

```
  discriminate H.
```

```
Qed.
```